CS 320 Project Two: Summary and Reflections Report

Gage Pigati

Southern New Hampshire University

Summary

Alignment to Requirements and Test Effectiveness

My unit tests were organized around the stated requirements for Contact, Task, and Appointment rather than simply checking whether constructors ran. ContactTest.java checks the ten-character ID limit, names of at most ten characters, exactly ten numeric phone digits, and an address of at most thirty characters. ContactServiceTest.java rejects duplicate IDs and null contacts. The service tests also exercise deletion and updates to all four permitted fields.

For Task, I tested its ten-character ID, twenty-character name, and fifty-character description. TaskTest.java checks invalid field values; TaskServiceTest.java checks duplicate IDs, deletion by ID, and updates that preserve the task ID. Appointment required a different focus: AppointmentTest.java rejects missing or past dates and invalid descriptions. AppointmentServiceTest.java tests unique additions and deletion. Because the appointment service only requires add and delete operations, I did not assume it needed update methods (Southern New Hampshire University, n.d.).

The final Project One ZIP contained 33 JUnit test methods: 13 Contact, 12 Task, and 8 Appointment. They covered valid actions, invalid inputs, and boundaries. My Module Four journal recorded 85.5% instruction coverage (460 of 538 instructions) and 21 passing Task and TaskService tests (Pigati, 2026). That historical result does not measure the final ZIP. Although pom.xml configures JaCoCo to require 80% line coverage during Maven verify, that configuration does not prove the final build passes. Instruction and line coverage measure different things, and both must be paired with meaningful assertions to demonstrate test effectiveness (Ammann & Offutt, 2016).

Technical Soundness and Efficiency

I used assertThrows to prove that validation fails when it should, rather than treating an absence of exceptions as sufficient. ContactTest.java checks rejected setter values and confirms that prior data remains unchanged. TaskTest.java makes the same kind of check for invalid task updates. ContactServiceTest.java tests that a failed batch update does not partially change a stored contact. For date integrity, AppointmentTest.java mutates both an original and a retrieved Date, then confirms the appointment still holds the expected time.

I kept the test suite efficient through short, focused tests and small object-creation helpers instead of extensive repeated setup. The Contact, Task, and Appointment service tests use helpers to create valid records. Each service test creates its own instance, so stored data does not leak between tests. Direct assertions make update failures easier to identify.

Reflection

Testing Techniques and Practical Uses

I used requirement-based JUnit unit testing with positive, negative, and boundary checks. Positive tests verified valid creation, retrieval, updates where supported, and deletion. Negative tests rejected null fields, duplicate IDs, nonnumeric phone numbers, and past dates. Boundary tests compared allowed limits with the first disallowed value, such as a 50-character appointment description versus 51 characters. These fast, repeatable tests are useful for scheduling and registration software, where validation errors could corrupt data. They cannot prove that connected application components work together.

I did not use formal integration, system, acceptance, performance, or static-review testing in these in-memory milestones. Integration tests would verify services communicating with a database or API, catching interface failures but requiring more setup. System testing would exercise a complete mobile booking workflow, detecting cross-component defects at a higher cost. Acceptance testing would compare booking and cancellation scenarios with customer expectations rather than only developer assumptions. Performance testing could examine response times under heavy request loads, revealing scalability issues. Static reviews could identify unclear requirements, insecure patterns, or design flaws without running code, but cannot establish runtime correctness. These techniques complement unit tests before deployment (Ammann & Offutt, 2016).

Caution, Bias, and Discipline

I tested cautiously because validation and service operations interact. A duplicate ID must not replace a record, and failed multi-field updates must not change only some fields. ContactServiceTest.java verifies that an invalid phone blocks changes to other contact fields. Appointments present another risk because Java's Date is mutable. Appointment.java uses defensive copies; AppointmentTest.java confirms changing an original or returned Date cannot alter the stored date. These cases show why examining methods without their interactions can miss defects.

Testing my own code introduces confirmation bias: I might check only paths I expect to pass. To reduce that risk, I used written requirements and deliberately supplied invalid data. ContactTest.java tries null values, incorrect phone lengths, letters, and spaces rather than only a valid number. Peer review and final-build coverage results would offer additional independent evidence.

Disciplined testing prevents shortcuts from becoming technical debt. Removing a failing test or loosening validation may make a build appear successful while leaving a customer-facing defect. I would rerun JUnit and JaCoCo after meaningful changes, record coverage by class, and add regression tests for fixed defects. If the services moved from in-memory maps to a database, I would retain validation tests and add integration checks for persistence and error handling. Maintaining these habits costs less than repairing failures after release.

References

Ammann, P., & Offutt, J. (2016). Introduction to software testing (2nd ed.). Cambridge University Press. https://doi.org/10.1017/9781316771273

Pigati, G. (2026). CS 320 Module Four journal [Unpublished course assignment]. Southern New Hampshire University.

Southern New Hampshire University. (n.d.). CS 320 Project One guidelines and rubric [Course materials].